HRIS Dev Workflow — Frontend & Backend (Complete Flow)

Versi: 1.0
Client: Atar
Tanggal: 2025-08-23

Ringkasan singkat

Dokumen ini menjelaskan alur kerja (workflow) pengembangan end-to-end untuk frontend (Next.js 14) dan backend (NestJS 10), termasuk branching model, CI/CD pipeline, quality gates, deployment strategy, database migration policy, release process, observability & rollback. Di bagian kedua terdapat daftar semua modul utama (fungsi), detail tanggung jawab tiap modul, API/event contracts utama, dan relasi antar modul.

Dokumen ditulis dengan fokus pada determinism (versi dikunci), safety untuk payroll/financial, dan minimal ambiguous decisions.

---

1. Branching model & developer flow

Model utama: trunk-based development with short-lived feature branches.

1. **Trunk/main** — always releasable, protected (CI green + approvals).
2. **Develop (opsional)** — boleh dipakai jika tim mau staging branch (recommended for >8 devs).
3. **Feature branches**: `feature/{ticketId}-{short-desc}` (e.g. `feature/HRIS-123-attendance-clockin`). Lifespan ≤ 7 hari.
4. **Hotfixes**: `hotfix/{short}` → langsung ke main, tag semantic version.
5. **Release branches**: only if necessary for long stabilization (`release/vX.Y`). Prefer fast trunk releases over long release branches.

PR rules - PR target: `develop` or `main` depending on flow.
- Must pass: lint, typecheck, unit tests, contract tests (if modifies API), security scan, dependency scan, and at least 1 code owner approval for backend / 1 UI reviewer for frontend.
- Feature PR description must include: purpose, API contract changes, migration steps (if DB), and acceptance checklist.

Local dev - Branch from trunk: `git checkout -b feature/HRIS-123-desc`
- Create API contract (OpenAPI minor change) or mock with MSW (frontend)
- Implement backend service + migration (if needed)
- Add unit tests + contract tests

- Run local e2e (Playwright) for critical path
- Push + open PR.

---

2. CI/CD Pipelines (high level)

Platform preferred: GitHub Actions (examples below use this), can be ported to GitLab CI / Azure DevOps.

Pipeline stages (common)

1. **PR pipeline** (runs on PR):
2. Checkout, cache node/pnpm
3. Install deps (`pnpm i --frozen-lockfile`)
4. Lint, typecheck
5. Unit tests (Jest)
6. Contract tests (Pact / OpenAPI validator) if API or DTO changed
7. Security scans: Snyk/Trivy (deps + container images)
8. Build (frontend SSR build & backend ts compile)
9. Artifact publish (store build artifacts for staging deploy)
10. **Merge pipeline (main)**:
11. All PR checks re-run
12. DB migration plan (generate migration; do not auto apply to prod)
13. Deploy to **staging** (k8s or platform) with blue/green or ephemeral preview
14. Run smoke tests & e2e
15. Auto-approve signoffs (QA) sets release candidate
16. **Release pipeline** (manual trigger):
17. Tag semantic version (vX.Y.Z)
18. Run DB migration job (pre-migration backup & lock)
19. Deploy to production using canary (10% → 50% → 100%)
20. Run post-deploy smoke tests and run critical business checks (payroll simulation for payroll deploys)
21. Monitor metrics for 30m; auto-rollback on critical errors or manual rollback.

Checks required before merge to main: - 100% of critical unit tests pass (no flakiness).
- Contract tests green.
- Security scan: no critical vulnerabilities added.
- If DB migration included → migration must be backward-compatible or flagged for release window.

Deployment targets - dev (developer preview) — ephemeral review apps for frontend, dev backend container.

- staging (integration) — representative production infra & data subset.
- prod — live.

---

3. DB Migration & Schema change policy

Principles: backward compatibility, zero-downtime deployments, reversible migrations where possible.

1. **Additive changes (safe)**: add column (nullable or with default) — permitted without release window.
2. **Non-additive/destructive**: drop columns, rename fields, change type — requires 2-step migration:
3. Step A: Introduce new fields & write dual-writes in app code (read old fallback)
4. Step B: Backfill data & migrate reads to new field (monitor)
5. Step C: Remove old fields in subsequent release (with signoff).
6. **Large backfills**: run as background job with throttle; not part of request path.
7. **Migration execution**: migrations applied via controlled job in CI/CD release pipeline (not by app on boot) with pre-backup, and migration lock to prevent concurrent deploys.

Prisma specifics - Use `prisma migrate dev` for local; `prisma migrate deploy` in release job.
- Keep migration scripts in repo and visible on PR.

---

4. API Contract & Compatibility rules

- **Versioning**: path versioning `/api/v1/...`. Minor non-breaking changes allowed under same version. Breaking changes require new major `v2` and migration guide.
- **Contract tests**: Backend must publish OpenAPI JSON; frontend runs contract tests (or Pact) against mock server or the staging API.
- **Deprecation**: Mark fields `deprecated` in OpenAPI with removal date & Sunsetting header on responses.

---

5. Frontend flow (Next.js 14 specific)

Dev workflow 1. Create feature branch.
2. Prototype UI in Storybook (component first).
3. Build pages using App Router / server components rules: server components for data-fetch; client components for interactive logic.
4. Use React Query (TanStack) for data fetch & caching.
5. Mock API with MSW for offline dev/unit tests.
6. Add visual regression test (Chromatic or Playwright snapshot).
7. Unit tests (Vitest or Jest) + e2e Playwright for critical flows.
8. PR checks run.

CI/CD for frontend - PR: lint, typecheck, unit tests, storybook build, visual test.
- Merge to main: build, prerender pages, test, publish artifact, deploy to Vercel or k8s.
- Preview apps: auto deploy preview on PR (Vercel or ephemeral).

Performance gating - Lighthouse check on key pages.
- P95 LCP target $\leq$ 2.5s (check on staging).

Secrets & env - Use environment variables per environment; do not expose secrets to preview apps (mask them).

---

6. Backend flow (NestJS 10 specific)

Dev workflow 1. API contract (OpenAPI) updated first if surface changes.
2. Implement service, DTOs & validators (class-validator).
3. Add Prisma schema + migration if DB changes.
4. Unit tests (Jest) + integration tests (Supertest).
5. Contract tests and smoke tests.
6. PR checks run.

CI/CD for backend - PR: lint, typecheck, unit & integration tests, OpenAPI validation, container build + image scan.
- Merge main: publish container image to registry, deploy to staging.
- Release: run `prisma migrate deploy` via a controlled job before deploying app containers if migration required.

Runtime concerns - Use readiness/liveness probes.
- Graceful shutdown handling to finish inflight jobs (esp. payroll jobs).

---

7. Quality gates & monitoring

Pre-merge: lint, typecheck, unit tests, contract tests.
Pre-release: staging smoke + E2E + security scan.
Post-release: health checks, business validation (sample payroll run), metrics monitoring (error rate, latency), and SLOs.

Automated rollback triggers - Error rate > 3% for 10 minutes.
- Increase in latency > 2x baseline for 10 minutes.
- Payroll simulation mismatch beyond threshold.

---

8. Critical release rules (Payroll & Finance modules)

Payroll is business-critical — special rules apply: 1. **Release window**: Payroll runs require scheduled window (non-business hours); DB backup & verification before deploy.
2. **Approval**: Release must be authorized by Finance + DevOps on call.
3. **Dry-run**: Must pass payroll dry-run on staging with sample dataset and reconciliation.
4. **Feature flags**: Use feature flags to gate changes to calculation logic; toggleable rollback.
5. **Audit**: Every payroll run must be logged & checksumed; keep artifacts for at least 7 years.

---

9. Modules — list & detailed responsibilities

Untuk tiap modul: singkat deskripsi, owner data (who owns source of truth), API responsibilities, events produced/consumed, UI surface.

> **Central rule**: `Employee` & `Organization` adalah sumber kebenaran master data — semua modul mengonsumsi, hanya Core HR menulis (create/update) kecuali proses onboarding yang menulis employee on `hired`.

9.1 Core HR (Master Data)

- **Deskripsi**: Menyimpan data personal, employment, document vault, lifecycle events.
- **Data owner**: Core HR service (single source of truth).
- **APIs**: CRUD employee, org structure, documents (upload / download presigned URL).
- **Events**: `employee.created`, `employee.updated`, `employee.terminated`.
- **UI**: Employee profile, org chart, documents.

Relasi: dipakai oleh Payroll, Benefits, Attendance, Performance, Asset, Recruitment.

9.2 Organization (Org & Position)

- **Deskripsi**: structure hierarki, cost centers, grades, positions.
- **APIs**: get org tree, update positions, assign cost center.
- **Events**: `position.created/updated`.

Relasi: Employee has position/grade; Payroll uses grade for pay rules; Finance uses cost center.

9.3 Recruitment / ATS

- **Deskripsi**: job posting, application pipeline, interview scheduling.
- **Owner**: ATS service; writes to `candidate` domain; on hire it emits `candidate.hired` → triggers Core HR onboarding workflow.

- **Events**: `candidate.applied`, `candidate.hired`.

Relasi: On hire → Core HR → Employee entry creation; assets provisioning.

9.4 Onboarding / Offboarding

- **Deskripsi**: Task checklists, provisioning (SCIM), probation tracker.
- **Events**: `onboarding.started`, `onboarding.completed`, `offboarding.started`, `offboarding.completed`.

Relasi: Listens to `candidate.hired` or `employee.created`.

9.5 Attendance & Scheduling

- **Deskripsi**: Clock-in/out, devices gateway, roster, timesheets, offline sync.
- **APIs**: ingest events, query timesheet, roster management.
- **Events**: `attendance.event.ingested`, `timesheet.updated`.

Relasi: Payroll consumes `timesheet` for earnings & overtime; HR uses for absence reasoning.

9.6 Leave & Absence

- **Deskripsi**: Policies, requests, approvals, accruals.
- **APIs**: request leave, balance query, approve/reject.
- **Events**: `leave.request.created`, `leave.request.approved`, `leave.balance.updated`.

Relasi: Payroll reads approved leaves to prorate pay; Attendance reconciles overlapping records.

9.7 Payroll (Finance critical)

- **Deskripsi**: Salary calc, tax (PPh21), BPJS, THR, deductions, payslip generation, bank file.
- **APIs**: run payroll, simulate, fetch payslip, export bank file.
- **Events**: `payroll.run.started`, `payroll.run.completed`, `payslip.generated`.

Relasi: Requires Core HR, Attendance (timesheet), Leave, Benefits & Claims, Organization (cost center) and Asset for allowances.

9.8 Benefits & Claims

- **Deskripsi**: Insurance enrollment, reimbursements, claims flow.
- **Events**: `claim.submitted`, `claim.approved`.

Relasi: Payroll reads benefits contribution; Finance handles reimbursements.

9.9 Performance & Goals

- **Deskripsi**: OKR/KPI, reviews, 360 feedback, calibration.

- **APIs**: set goals, submit review, fetch ratings.
- **Events**: `goal.created`, `review.completed`.

Relasi: Compensation decisions reflect here (manual or automated tie to payroll).

9.10 Learning (LMS light)

- **Deskripsi**: Course catalog, assignment, completion, certification.
- **Events**: `course.completed`.

Relasi: On completion, may trigger performance points or eligibility for role change.

9.11 Expenses & Travel

- **Deskripsi**: Claim submission, travel requests, per-diem calc, multi-currency.
- **APIs**: submit claim, approve, export to payroll/finance.
- **Events**: `expense.submitted`, `expense.approved`.

Relasi: Finance integrates with Payroll to process reimbursements.

9.12 Asset Management

- **Deskripsi**: lifecycle of hardware & access cards, depreciation.
- **APIs**: assign asset, return asset, service ticket.
- **Events**: `asset.assigned`, `asset.returned`.

Relasi: Onboarding consumes asset assignment; Offboarding triggers asset return.

9.13 Communication & Notifications

- **Deskripsi**: targeted announcements, notification routing (email, WA, Slack), scheduling.
- **APIs**: create announcement, subscribe.
- **Events**: `announcement.published`, `notification.sent`.

Relasi: All modules trigger notifications (leave approvals, payslip ready, onboarding tasks).

9.14 Analytics & Reporting (BI)

- **Deskripsi**: Dashboards, report builder, ETL to data warehouse.
- **APIs**: snapshot exports, query endpoints for dashboards.
- **Events**: snapshot exports, CDC stream.

Relasi: Pulls from all modules; owned by Data team. Writes back insights to Performance/HR as needed.

9.15 Integration Layer & Gateway

- **Deskripsi**: Adapters for bank, e-sign, SSO (Keycloak), attendance devices, accounting.

- **APIs**: adapters abstract remote vendor specifics.
- **Events**: `external.sync.success/fail`.

Relasi: Integrates with Payroll, Onboarding, Documents, Attendance.

9.16 Admin / System (RBAC, Config)

- **Deskripsi**: tenant, roles, policy engine, system settings (tax rates, BPJS rates).
- **APIs**: CRUD config, role mapping.
- **Events**: `config.updated`.

Relasi: Read by Payroll & other modules for rule engines.

9.17 AuthN/AuthZ

- **Deskripsi**: Keycloak realm, clients, claim mapping; token verification service.
- **Relasi**: All services validate tokens; SCIM used by Onboarding for provisioning.

---

10. Eventing & Data Contracts

Event bus: Kafka primary; events follow strict schema (Avro/JSON Schema) with versioning.

- Event payload contains `meta` (tenant_id, entity_id, correlation_id, causation_id, version) and `data`.
- Producers must publish events for state changes (employee, payroll run, attendance ingestion, etc.).
- Consumers must be tolerant to new fields in `data` (forward compatible).

Example: `employee.created` (see SRS Lampiran C).

---

11. Ownership, SLAs & SLIs per module

- Each module has an owner team (or person). Owner maintains: API doc, contract tests, runbooks, SLOs.
- **SLA/SLI examples**:
- Attendance ingestion P95 latency < 10s.
- Payroll run for 5k emp < 20 min.
- Employee CRUD API availability 99.9%.

---

12. Diagrammatic flows (textual sequences)

(Two sequences; can later be converted to sequence diagrams.)

12.1 Feature dev: Add `Clock-In` endpoint

1. Ticket created (product).
2. Backend: design DTO + OpenAPI change, add controller + service + prisma migration (add attendance_event table), unit tests.
3. Frontend: story + component + mobile offline queue logic (MSW + hooks).
4. PR backend + PR frontend; contract test ensures shape.
5. CI runs; artifacts created; preview app uses mock backend.
6. Merge to staging; run integration tests with device gateway emulator.
7. QA signoff; schedule deploy to prod (non-payroll window).
8. Monitor ingestion metrics & errors for 1 hour.

12.2 Release payroll calculation change (safe rollout)

1. Dev implements new calculation behind feature flag.
2. Run extensive unit & integration tests (edge cases) + simulation on staging data.
3. Finance runs dry-run and verifies reconciliation.
4. Deploy to prod with flag off; run scheduled payroll simulation with small sample.
5. Toggle flag for 5% users (canary) → monitor key metrics & reconcile.
6. Expand to 50% → 100% if OK.
7. Remove old code per deprecation plan.

---

13. Deliverables per sprint (recommended)

- Code: feature branches & merged PRs.
- Tests: unit + integration + contract + e2e smoke.
- Docs: OpenAPI updated, runbook, migration notes.
- Infra: helm chart or k8s manifest updates.
- QA: test cases executed & signoff.

---

14. Runbooks & playbooks (must-have list)

- Deploy payroll release playbook (pre-backup, migration step, verification).
- Rollback plan (how to revert DB + app + feature flags).
- Incident triage (severity matrix, alerting, on-call rotation).
- Data breach handling.

---

15. Next actionable items (prioritized)

1. Create GitHub Actions templates for frontend & backend (PR + release).

2. Implement OpenAPI publishing + contract tests scaffold.
3. Implement Prisma multi-tenant middleware pattern.
4. Build Storybook + MSW templates for frontend.
5. Create payroll release runbook and gating automation.

Appendix: Glossary & Conventions

- `tenant_id` must be present in events & API calls.
- UUID standard: v7 (if available) or v4.
- Timestamps in events: ISO 8601 UTC.
- Semantic versioning for services & API major versioning.

Dokumen ini dibuat untuk jadi blueprint actional — jika mau, gue bisa lanjut generate: - GitHub Actions files for both apps, or - Prisma middleware for tenant injection, or - Storybook + MSW starter for the frontend.

Pilih salah satu next task dan gue langsung gas bikin file + instruksi.